

A Lightweight Detection Algorithm For Collision-Optimized Divide-and-Conquer Attacks

Changhai Ou^{ID}, Siew-Kei Lam^{ID}, Chengju Zhou^{ID}, Guiyuan Jiang^{ID}, and Fan Zhang^{ID}

Abstract—By introducing collision information into divide-and-conquer attacks, several existing works transform the original candidate space, which may be too large to enumerate, into a significantly smaller collision space, thus making key recovery possible. However, the inefficient collision detection algorithms and fault tolerance mechanisms make them time-consuming and their success rate low. Moreover, they may still leave very huge chain spaces that makes it difficult for key recovery. In this article, we exploit collision attack to optimize Template Attack (TA), and propose a Lightweight Collision Detection (LCD) algorithm. The proposed method exploits a jump detection mechanism to efficiently reduce the repetitive collision detections on chains with the same prefix sub-chains. We then introduce guessing theory to reorder the collision detection of the sub-keys according to their guessing lengths, and provide us with an evaluation tool. Finally, we design a highly efficient fault tolerance mechanism for our LCD to allow flexible thresholds adjustment, and further optimize sieving mechanism to efficiently extract the best chains with the largest number of collisions. Experimental results fully demonstrate LCD's superiority.

Index Terms—Collision detection, LCD, collision attack, template attack, fault tolerance, key enumeration, side-channel attack

1 INTRODUCTION

SECRET information will leak through side-channels such as Timing [11], power consumption [25], electromagnetic [8], cache patterns [13] and acoustic [9] when cryptographic algorithms are implemented on devices. Side-channel attacks (SCAs) can be launched in two ways: divide-and-conquer and analytical. The former like Correlation Power Analysis (CPA) [4] and Template Attack (TA) [6], is performed on each sub-key like AES-128 and conquer them one by one. The latter like collision attacks [14], [21], conquers the key by solving a system of equations. It exploits more information compared to the divide-and-conquer manner, but is more complex. Key enumeration strategies [12], [20], which enumerate the full-key candidates from the most possible one to the least possible one, are exploited to optimize the key recovery if the divide-and-conquer attacks do not rank the sub-keys as their best candidates. However, they are limited by the computing power of the attacker, and can only be exploited in scenarios where cryptographic implementations are “practically insecure” (for which the leakage allows for key enumeration).

In this paper, our goal is neither to improve the collision attack like [5], [10], [26], nor to enhance the key enumeration

schemes. Instead, we aim to exploit collision information from collision attacks like Correlation-enhanced Collision Attacks (CCA) [14] to optimize the divide-and-conquer attacks like TA, and transform their huge candidate space to collision chain space. In this case, the candidates dissatisfying the collision conditions are discarded, thus key recovery can still be feasibly applied to the remaining chain space that is much smaller than the one provided by divide-and-conquer attack (see experiments in Section 6). We called these combined collision attacks, Collision-Optimized Divided-and-Conquer Attacks (CODCAs) in [16]. They are independent of specific distinguishers used, and their collision detection algorithms play a very important role in performance. Related works will be described in the next subsection before introducing our contributions.

1.1 Related Works

The idea of exploiting collision information to optimize the divide-and-conquer attacks was first given in [3]. For example, the divide-and-conquer attack TA can be optimized by the collision attack like CCA. The combined attack exploits key-related information from the outputs of two attacks, and aims to recover the full key from them under given thresholds. This means that only a very small part of candidates within thresholds are considered. The proposed CODCAs in this paper can exploit collision information to further reduce the candidate space. Let κ_j denote the j th sub-key, and τ_κ denote the threshold, which means only the τ_κ best candidates of each sub-key will be considered. Similarly, we also set a threshold τ_d for CCA, which means only the τ_d best candidates for each XOR value $\delta_{j_1, j_2} = \kappa_{j_1} \oplus \kappa_{j_2}$ of two sub-keys κ_{j_1} and κ_{j_2} are considered. A collision happens if a pair of candidates of two sub-keys and their corresponding δ candidate are within thresholds simultaneously, and several collisions form a chain. For example, a chain consisting of 3 sub-keys

- Changhai Ou, Siew-Kei Lam, Chengju Zhou, and Guiyuan Jiang are with the Hardware and Embedded Systems Lab (HESL), School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798. E-mail: {chou, assklam, zhou0271, gyjiang}@ntu.edu.sg.
- Fan Zhang is with the School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University, Hangzhou, Zhejiang 310027, China, and also with the Alibaba-Zhejiang University Joint Institute of Frontier Technologies, Hangzhou, Zhejiang 310027, China. E-mail: fanzhang@zju.edu.cn.

Manuscript received 14 Jan. 2020; revised 29 Apr. 2020; accepted 1 June 2020.
Date of publication 16 June 2020; date of current version 8 Oct. 2020.
(Corresponding author: Fan Zhang.)
Recommended for acceptance by A. Awad.
Digital Object Identifier no. 10.1109/TC.2020.3002795

κ_1 , κ_2 and κ_3 is formed if a candidate of κ_2 simultaneously collides with a candidate of κ_1 and a candidate of κ_3 . In this case, each chain includes several sub-keys and has a freedom variable. If the full key is divided into several chains, they have to enumerate the freedom variables simultaneously, which is too time-consuming. Therefore, the authors in [3] tried to find a chain named Test of Chain (TC) that included all 16 sub-keys of AES-128 algorithm.

Wang *et al.* proposed a new CODCA named Fault Tolerant Chain (FTC) in [24], which aimed to find collisions between the first sub-key and the remaining 15 sub-keys. FTC can set a fixed threshold τ_k for these sub-keys. It detects collision between each guessing value of κ_1 and the other 15 sub keys under a given threshold τ_d independently, and its complexity is about $15 \cdot \tau_k^2 \cdot \tau_d$. However, the remaining candidates of these 15 sub-keys under FTC are still independent. Due to the insufficient utilization of collision information, they can only be recovered by traditional key enumeration, which also makes the later key recovery very time-consuming. Moreover, some sub-keys may also fail to collide with κ_1 . FTC enumerates their candidates one by one. Therefore, the difficulty of key recovery increases dramatically.

Group Collision Attack (GCA) [17] divides the full-key into several big non-overlapping “groups” with same number of sub-keys and exploit as many collisions as possible to construct sub-chains within each group. It continuously splices long chains from short chains. More than twice as many collisions are utilized compared to TC and FTC. However, the location of these exploited collisions is fixed and GCA does not include any fault tolerance strategy. Any sub-key or collision value out of thresholds τ_k or τ_d may lead to failure of key recovery. In addition, harsher collision conditions, which require more collisions, are also more difficult to be satisfied. Therefore, the number of remaining chains in GCA is smaller compared to TC and FTC. Hence, GCA is suitable for scenarios with large thresholds.

Our work does not aim to improve key enumeration or rank estimation. Instead, we aim to introduce collision information from a collision attack into the divide-and-conquer attack to transform the original huge space to a much smaller chain space. This can make key recovery easier. If only a small part of candidates with small probabilities are removed after transformation, the CODCAs will not significantly reduce the probability of key recovery. Group Verification Chain (GVC) in [18] exploited the collisions between a sub-key and other sub-keys to re-rank its candidates (according to the number of collisions), which makes the recovery of each sub-key easier. However, GVC neither changes the output scores or probabilities of candidates, nor deletes a part of them. In other words, it does not change the rank of key. The multiple-differential mechanism for CODCAs in [16] builds chains from several sub-keys, then exploits them to vote for the remaining sub-keys one by one. These sub-keys are still independent of each other, thus facilitating the later key enumeration. It does not splice the long chains from the short ones like TC, FTC and GCA, so it is not considered in this work. There were also several algorithms like CCA in [26] that relies on sieving the candidates. These works are orthogonal to this paper, and hence we do not compare with them.

1.2 Our Contributions

This paper proposes an efficient Lightweight Collision Detection (LCD) algorithm for CODCAs. Our main contributions are as follows:

- (i) We propose a novel Lightweight Collision Detection (LCD) algorithm for CODCAs. It exploits an efficient jump collision detection mechanism to avoid a large number of repetitive collision detections when building long chains from the short ones, and significantly reduces the time consumption.
- (ii) We exploit guessing theory to reorder the collision detection of the sub-keys according to their guessing length (i.e., number of candidates within τ_k) under a fixed probability, thus optimizing our LCD. Guessing theory also provides CODCAs with a very good evaluation tool.
- (iii) We introduce a highly efficient fault tolerance mechanism into our LCD algorithm, which enables a certain number of collision values to exceed the thresholds. We further exploit sieving mechanism to flexibly extract the chains with almost the largest number of collisions, thus guaranteeing that the best chains are within the current chain space. The correct key is usually one of these chains.

Experimental results show that our LCD algorithm can achieve a significantly higher success rate than the existing schemes. The remaining smaller chain space also makes the future key recovery easier compared to the existing schemes.

1.3 Organization

The rest of this paper is organized as follows: Measurement setups, Template Attack (TA), CCA and Minkowski Distance enhanced Collision Attack (MDCA) are introduced in Section 2. The existing CODCAs like TC, FTC and GCA, are presented in Section 3. The difference in collision detection between them and classic collision attack is also given in this section. Our LCD is presented in Section 4. The fault tolerance strategies and guessing theory based optimization are discussed in Section 5. To alleviate the burden of LCD, an optimized sieve mechanism to extract the chains with the largest number of collisions are also described in this section. Experiments on an AT89S52 micro-controller are presented in Section 6 to illustrate the superiority of our LCD. Finally, we conclude this paper in Section 7.

2 PRELIMINARIES

2.1 Experimental Setups

We implemented the AES-128 algorithm on an AT89S52 micro-controller designed for SCAs using assembly language. The operating frequency of this chip is 12 MHz, and the shortest instructions take 12 clock cycles. Sampling rate of our Picoscope 3000 was set to 125 MS/s, thus each clock cycle includes about 10 samples. We acquired 100,000 power traces and performed classic CPA to select the time sample with the highest correlation coefficient for the first S-box, then extracted the well aligned samples of other 15 S-boxes. We then undertook our experiments using MATLAB R2016b on a HP desktop computer with 6 Intel(R) Xeon(R) E5-1650 CPUs, 16 GB RAM and a Windows 10 operating system.

2.2 Template Attack

The classic Template Attack (TA) [6] assumes that the attacker has the same device as the target device. He can collect any number of power traces from his own device to profile enough accurate leakage models to launch attacks. Taking the example of a device that follows the Hamming weight leakage model, the attacker encrypts n_j plaintexts $\mathbb{X}_j = (x^1, x^2, \dots, x^{n_j})$ having the same Hamming weight of their S-box outputs, and acquires n_j power traces $\mathbb{T}^j = (t^1, t^2, \dots, t^{n_j})$. He then exploits their interesting points [27] $(t^1, t^2, \dots, t^{n_j})$ to profile the mean:

$$\mathbf{m}^j = \frac{1}{n_j} \sum_{i=1}^{n_j} t^i \quad (1)$$

and covariance matrix:

$$\mathbf{C}^j = \frac{1}{n_j} \sum_{i=1}^{n_j} (t^i - \mathbf{m}^j)(t^i - \mathbf{m}^j)^T, \quad (2)$$

for templates $(\mathbf{m}^j, \mathbf{C}^j)$ ($0 \leq j \leq 8$) corresponding to Hamming weights j ($0 \leq j \leq 8$) in the profiling stage. Here the symbol “ T ” denotes matrix transposition. The probability of a power trace $\mathbf{t}$ (with interesting points t) sampled from targeted device generated by encrypting x having the same Hamming weight with template $(\mathbf{m}^j, \mathbf{C}^j)$ satisfies:

$$p(t|\mathbf{m}^j, \mathbf{C}^j) = \frac{e^{\frac{(\mathbf{m}^j - t) \cdot (\mathbf{C}^j)^{-1} \cdot (\mathbf{m}^j - t)^T}{2}}}{\sqrt{(2 \cdot \pi)^{|\mathbf{m}^j|} \det(\mathbf{C}^j)}}. \quad (3)$$

$|\mathbf{m}^j|$ represents the number of samples on template $\mathbf{m}^j$.

Zhang *et al.* proposed an optimized Template Attack in [27] exploiting the fact that the power leakages obtained from different power traces are independent. The match probabilities can be multiplied together to obtain the probability under each guessing sub-key. We further convert multiplication to addition using logarithm of probabilities, and normalize the probabilities $p_j = \{p_j^1, p_j^2, \dots, p_j^{|\mathbb{K}_j|}\}$ corresponding to candidates of κ_j as:

$$\sum_{i=1}^{|\mathbb{K}_j|} p_j^i = 1, \quad (4)$$

where p_j^i is the i th largest probability. The similar strategy was exploited in [7] to estimate the guessing entropy [22] in side-channel attacks.

2.3 Minkowski Distance Enhanced Collision Attack

Let κ_j denote the j th sub-key, and x_j denote the corresponding encrypted plaintext byte. An AES-128 linear collision [3] occurs if two S-boxes in the same AES encryption or several encryptions receive the same byte value as their inputs. The attacker finds a collision:

$$\text{Sbox}(x_{j_1} \oplus \kappa_{j_1}) = \text{Sbox}(x_{j_2} \oplus \kappa_{j_2}), \quad (5)$$

which can be optimized as:

$$\delta_{j_1, j_2} = \kappa_{j_1} \oplus \kappa_{j_2} = x_{j_1} \oplus x_{j_2}. \quad (6)$$

Here “ $\text{Sbox}(\cdot)$ ” denotes the look-up table operation. Correlation-enhanced Collision Attack (CCA) can be used to detect δ -s. It divides the power traces according to the plaintext byte value x_j from 0 to 255, and computes their means (denoted by $\bar{t}_j^{x_j}$ ($j = 1, 2, \dots, 16$)). In other words, $\bar{t}_j^{x_j}$ denotes the mean power consumption of plaintext byte value x_j of the j th sub-key. Taking the first two sub-keys for an example, CCA then computes the correlation coefficient between them:

$$\rho\left\{\left(\bar{t}_1^{x_1}, \bar{t}_2^{x_1 \oplus \delta_{1,2}}\right) \mid x_1 = 0, 1, 2, \dots, 255\right\} \quad (7)$$

under a guessing $\delta_{1,2}$.

It is noteworthy that CCA in [14] was designed for detecting flawed masking implementations like DPA *contest v4.1* [1]. It's performance is far inferior to the divide-and-conquer attack like CPA exploited in [24] and the improved TA [27] exploited in this paper. Therefore, it is necessary to set a large τ_d for TC, FTC and GCA to make more collision values δ -s fall within it, thus guaranteeing the successful detection. Therefore, to reduce the performance gap, we exploit our improved collision attack named Minkowski Distance enhanced Collision Attack (MDCA) given in [15]. Specifically, we use the second order of Minkowski Distance (i.e., Euclidean distance) in MDCA, and define the distance function d as:

$$d_{\kappa_1, \kappa_2} = \sqrt{\sum_{i=1}^n \sum_{j=1}^2 \left(t_j^i - m_j^{\text{Sbox}(x_j^i \oplus \kappa_j)} \right)^2}. \quad (8)$$

Here n denotes the number of power traces exploited in attacks, x_j^i denotes the j th byte of the i th encrypted plaintext and t_j^i denotes the corresponding interesting point. The mean power consumption can only be profiled based on the intermediate values rather than plaintexts. Therefore, $m_j^{\text{Sbox}(x_j^i \oplus \kappa_j)}$ denotes the profiled mean power consumption corresponding to intermediate value output by S-box receiving $x_j^i \oplus \kappa_j$ as its input. We extract the minimum distance for each $\delta_{1,2} = \kappa_1 \oplus \kappa_2$ under all possible guessing κ_1 and κ_2 , and the later attack of MDCA is similar to traditional CCA. It can be seen from the experimental results performed on randomly selected traces in Fig. 1 that the performance of traditional CCA under $\tau_d = 30$ is still far behind that of MDCA under $\tau_d = 1$.

3 EXISTING CODCAs

3.1 Collision Detection

The collision detection function in classic collision attacks [2] can be defined as:

$$\phi(x_{j_1} \oplus \kappa_{j_1}, x_{j_2} \oplus \kappa_{j_2}) = \begin{cases} 1, & \text{if } \mathcal{D}(t_{j_1}, t_{j_2}) \leq \tau_{po} \\ 0, & \text{else} \end{cases} \quad (9)$$

The function $\mathcal{D}$ denotes the similarity of two traces (e.g., Euclidean distance), and τ_{po} is the corresponding threshold. In principle, the leakage of the same intermediate value in two look-up table operations has a high similarity. However, collision detection in CODCAs is quite different from

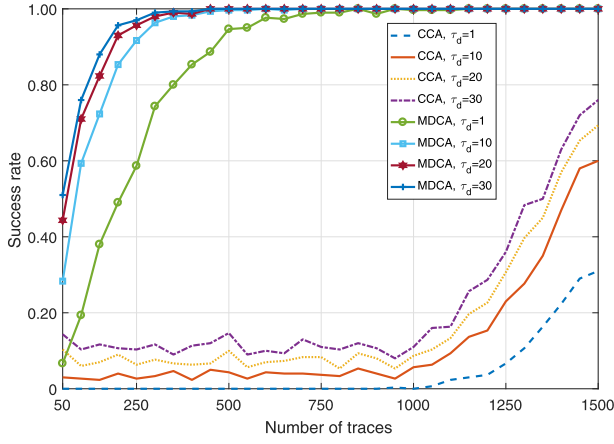


Fig. 1. Success rates of the traditional CCA and MDCA under different thresholds τ_d -s and different number of traces.

this. The collision detection algorithms like FTC, rank the candidates of $\delta_{1,2}$ according to their distances in MDCA. Then, they exploit these ranked collision candidates to optimize the candidate space of divide-and-conquer attacks like the optimized TA in Section 2.2. Let ξ_j^i ($1 \leq i \leq 256$, $1 \leq j \leq 16$) denote the i th best candidate of the j th sub-key. We only consider the best τ_κ candidates of each sub-key, the new collision detection function can be defined as:

$$\phi(\xi_{j_1}^{i_1}, \xi_{j_2}^{i_2}) = \begin{cases} 1, & \text{if } \xi_{j_1}^{i_1} \oplus \xi_{j_2}^{i_2} \in \{v_{j_1, j_2}^1, \dots, v_{j_1, j_2}^{\tau_d}\} \\ 0, & \text{else} \end{cases} \quad (10)$$

Here $i_1, i_2 \leq \tau_\kappa$, and $v_{j_1, j_2}^{i_3}$ ($1 \leq i_3 \leq \tau_d$) is the i_3 -th best candidate of δ_{j_1, j_2} . The Eq. (10) means the XOR value of candidates of two sub-keys within τ_κ , should be one of the best τ_d candidates in the MDCA given in Section 2.3. Here let $\xi_{j_1}^{i_1} \leftrightarrow \xi_{j_2}^{i_2}$ denote this pair of collision, a candidate of $\kappa_{j_1} \leftrightarrow \kappa_{j_2}$.

3.2 Test of Chain

Test of Chain (TC) [3] was the first CODCA. It introduces collision information into divide-and-conquer attacks such as TA and CPA, so as to build the relationship between sub-keys, and turn the huge candidate space into a much smaller chain space. It aims to find a chain from the first sub-key to the last sub-key. Taking AES-128 for an example, TC detects $\kappa_1 \leftrightarrow \kappa_2, \kappa_2 \leftrightarrow \kappa_3, \dots, \kappa_{14} \leftrightarrow \kappa_{15}$ and $\kappa_{15} \leftrightarrow \kappa_{16}$ in turn, and the newly considered sub-key is continuously spliced to obtain longer chains during detection. Finally, TC obtains chains that contains 16 sub-keys each. The sub-keys can form a chain and be left in the final remaining space when τ_d and τ_κ are set properly. Wang *et al.* introduced CCA into CPA to optimize its key candidate space and improved attack performance in [24]. In fact, as long as the performance of collision attack in CODCAs is not very different from the divide-and-conquer attack, the collision information can be well exploited to optimize its key recovery.

3.3 Fault Tolerant Chain

FTC [24] tries to find the collisions $\kappa_1 \leftrightarrow \kappa_2, \kappa_1 \leftrightarrow \kappa_3, \dots, \kappa_1 \leftrightarrow \kappa_{15}$ and $\kappa_1 \leftrightarrow \kappa_{16}$ between the first sub-key and the remaining 15 sub-keys to optimize the divide-and-conquer attacks. If there are $n_j^{\theta_j}$ ($2 \leq j \leq 16$) candidates of sub-key κ_j

left when considering a candidate value of κ_1 , then the number of remaining chains starting from this candidate is

$$\prod_{j=2}^{16} n_j^{\theta_j}. \quad (11)$$

FTC then traverses all candidates of κ_1 within threshold τ_κ and obtains the remaining chain space.

If none of the candidates of a sub-key meets the collision conditions, FTC enumerates it from the most possible one to the least possible one. The enumeration here is performed on each sub-key independently, which is different from the classic key enumeration in [20], [23]. The complexity reaches $2^8 \cdot (2^8)^n \cdot \binom{15}{n}$ if this fault happens on n sub-keys as given in [24]. The authors just considered the performance of TC and FTC under the case $\tau_d = 1$, where this fault often occurred. However, this is insufficient to consider the simple case of $\tau_d = 1$ to reduce candidate space in CODCAs, as more collision information needs to be considered. In other words, we need to consider the case where τ_d is significantly greater than 1, and even greater than τ_κ . There are $\tau_d \cdot \tau_\kappa$ collision detections in total, the probability that none of the candidates of a sub-key within τ_κ meets the collision conditions becomes very small.

3.4 Group Collision Attack

GCA [17] is different from TC and FTC. It divides 16 sub-keys into several big “groups” and aims to fully utilize collisions to reduce the number of sub-chains within each of them. It detects collisions $\kappa_1 \leftrightarrow \kappa_2, \kappa_1 \leftrightarrow \kappa_3$ and $\kappa_2 \leftrightarrow \kappa_3$ and obtains $\kappa_1 \leftrightarrow \kappa_2 \leftrightarrow \kappa_3$ including 3 pairs of collisions. Similarly, GCA checks whether the two candidate values of κ_2 and κ_3 are the same after detecting $\kappa_2 \leftrightarrow \kappa_3 \leftrightarrow \kappa_4$. If so, the first group $\kappa_1 \leftrightarrow \kappa_2 \leftrightarrow \kappa_3 \leftrightarrow \kappa_4$ is obtained by splicing. GCA detects $\kappa_3 \leftrightarrow \kappa_4 \leftrightarrow \kappa_5 \leftrightarrow \kappa_6$ and $\kappa_5 \leftrightarrow \kappa_6 \leftrightarrow \kappa_7 \leftrightarrow \kappa_8$ in the same way. A longer chain $\kappa_1 \leftrightarrow \kappa_2 \leftrightarrow \dots \leftrightarrow \kappa_8$ can be obtained if the candidates of κ_3 and κ_4 of $\kappa_3 \leftrightarrow \kappa_4 \leftrightarrow \kappa_5 \leftrightarrow \kappa_6$ are within the first group $\kappa_1 \leftrightarrow \kappa_2 \leftrightarrow \kappa_3 \leftrightarrow \kappa_4$ and its candidates of κ_5 and κ_6 are also within the second group $\kappa_5 \leftrightarrow \kappa_6 \leftrightarrow \kappa_7 \leftrightarrow \kappa_8$. GCA exploits 32 pairs of collisions significantly more than 15 pairs in TC and FTC. Therefore, GCA is suitable for the case of large τ_d . However, GCA has no fault tolerance strategy. It exploits more collisions compared to TC and FTC under the same τ_κ and τ_d , which leads to smaller chain space and smaller probability to recover the key.

3.5 Typical Example

We exploit 100 power traces to profile each Hamming weight template in the optimized TA introduced in Section 2.2 and template for each intermediate value in (the second order) MDCA given in Section 2.3. We then randomly extract 240 power traces to launch them. Here τ_κ and τ_d are set to 10 and 15, respectively. The ranked candidates of the first 3 sub-keys and their corresponding collision candidates of δ -s within thresholds are shown in Tables 1 and 2. We then exploit FTC and GCA to detect the collision chains. The chains $\kappa_1 \leftrightarrow \kappa_2, \kappa_2 \leftrightarrow \kappa_3$ and $\kappa_1 \leftrightarrow \kappa_3$ including two sub-keys are shown in Table 3, of which the number is 6, 8 and 7, respectively.

GCA divides 16 sub-keys of AES-128 algorithm into several groups, and reduces their candidates one by one. It

TABLE 1
The Ranked Candidates of 3 Sub-Keys Within τ_k
in the Optimized TA

κ_1	κ_2	κ_3
212	36	17
54	122	176
32	72	210
100	226	18
188	81	132
218	195	76
6	153	246
176	182	122
46	236	42
175	95	192

TABLE 2
The Candidates of δ -s within τ_d in MDCA

$\delta_{1,2}$	$\delta_{1,3}$	$\delta_{2,3}$
250	107	206
77	197	253
101	176	95
229	171	233
41	110	35
156	200	136
131	128	38
19	131	205
46	201	132
137	173	32
63	198	72
73	47	254
76	229	139
225	6	83
187	151	96

TABLE 3
The Collisions in Optimized TA Combined With MDCA

$\kappa_1 \leftrightarrow \kappa_2$		$\kappa_1 \leftrightarrow \kappa_3$		$\kappa_2 \leftrightarrow \kappa_3$	
212	72	212	17	122	132
212	153	212	210	72	192
54	122	212	18	195	42
176	81	188	17	153	17
176	153	188	210	153	18
175	226	188	122	236	17
		218	18	236	18
		6	192		

then combines the remaining sub-chains and obtains longer chains. According to this principle, it first splices $\kappa_1 \leftrightarrow \kappa_2$ and $\kappa_2 \leftrightarrow \kappa_3$, and gets a total number of 6 chains containing 3 sub-keys (as shown in Table 5). In fact, the collision $\kappa_1 \leftrightarrow \kappa_3$ was also detected in [17], and the two remaining chains are shown in Table 6, of which the bold one is correct. Compared to GCA, FTC exploits less collision information and removes fewer candidates. There are 6 chains left as shown in Table 4. The more advanced chains include more sub-keys and collisions are built on the short and simple sub-chains. As such, their number will be reduced rapidly. Therefore, the remaining collision detection will be much quicker, thus reducing the complexity of key recovery.

TABLE 4
Chains in FTC

κ_1	κ_2	κ_3
212	72	17
212	72	210
212	72	18
212	153	17
212	153	210
212	153	18

TABLE 5
Chains in TC Include $\kappa_1 \leftrightarrow \kappa_2$ and $\kappa_2 \leftrightarrow \kappa_3$

κ_1	κ_2	κ_3
212	72	192
212	153	17
212	153	18
54	122	132
176	153	17
176	153	18

TABLE 6
Chains Include $\kappa_1 \leftrightarrow \kappa_2$, $\kappa_1 \leftrightarrow \kappa_3$ and $\kappa_2 \leftrightarrow \kappa_3$

κ_1	κ_2	κ_3
212	153	17
212	153	18

4 LIGHTWEIGHT COLLISION DETECTION ALGORITHM

4.1 Collision Condition

Traditional CODCAs like TC, FTC and GCA, traverse the current collision chains set $\mathcal{R}_{j-1}$, detect the collision between them and the incoming sub-key κ_j one by one. They then add the chain satisfying the collision conditions to the new set $\mathcal{R}_j$. Therefore, they satisfy:

$$\mathcal{R}_j = \mathcal{R}_{j-1}(\kappa_1 \leftrightarrow \kappa_2 \leftrightarrow \dots \leftrightarrow \kappa_{j-1}) + \psi(\kappa_1 \leftrightarrow \kappa_2 \leftrightarrow \dots \leftrightarrow \kappa_{j-1}, \kappa_j). \quad (12)$$

Here $\psi(\cdot)$ is the collision detection function and it satisfies:

$$\psi(\kappa_1 \leftrightarrow \kappa_2 \leftrightarrow \dots \leftrightarrow \kappa_{j-1}, \kappa_j) = \phi(\kappa_{j-1}, \kappa_j) \quad (13)$$

in the existing TC. Since it only exploits the collision information between sub-key κ_{j-1} and κ_j . Similarly, only $\phi(\kappa_1, \kappa_j)$ is exploited in FTC. Here the definition of $\phi(\cdot)$ satisfies the Eq. (10).

The chain space dramatically grows when considering larger thresholds τ_k and τ_d . More harsher collision conditions that require more collisions to be satisfied will efficiently alleviate this. Here we exploit all collisions between any two sub-keys just like [15] and the collision detection function is defined as:

$$\psi(\kappa_1 \leftrightarrow \kappa_2 \leftrightarrow \dots \leftrightarrow \kappa_{j-1}, \kappa_j) = \sum_{i=1}^{j-1} \phi(\kappa_i, \kappa_j), \quad (14)$$

in our Lightweight Collision Detection (LCD) algorithm. Based on this, we will introduce its chain construction with a

highly efficient jump collision detection mechanism in Section 4.2, and its chain sieve in Section 4.3. For the convenience of readers, we give an example of our LCD in Section 4.4.

4.2 Chain Construction

Many chains in space $\mathcal{R}_{j-1}$ have the same “prefixes”, i.e., the candidates of their first several sub-keys are the same, since they are built from the same sub-chain (see Tables 4, 5 and 6). Obviously, if we detect the chains one by one when considering a coming sub-key, these “prefixes” will be repetitively detected. With the increase of thresholds τ_k and τ_d , the number of chains sharing the same prefixes will increase significantly, and this repetitive collision detection will make the CODCA time-consuming. Here, the chain construction procedure of our LCD is given in Algorithm 1 to alleviate this burden. LCD also includes a chain sieve mechanism and a very important fault tolerance mechanism, which will be introduced in Sections 4.3 and 5.

The inputs of LCD algorithm include the current chains $\mathcal{R}_{j-1}$ consisting of the first $j-1$ sub-keys, an array $\mathcal{F}$ to indicate the number of chains coming from the same prefixes, ranked candidates ξ_j of κ_j according to their probabilities and the corresponding threshold τ_{κ_j} , the number of collisions on each chain in array “numC”, and the minimum number of collisions to be satisfied by each chain under consideration τ_c . LCD algorithm outputs the new chains $\mathcal{R}_j$, array $\mathcal{F}$ and “numC” after finishing traversal. Let $\mathcal{R}_{j-1}^\beta$ denote the β -th chain in $\mathcal{R}_{j-1}$, $\mathcal{F}_\gamma$ denote the γ -th flag in $\mathcal{F}$, η_i denote the candidate of the i th sub-key on chain η and π denote the number of chains in $\mathcal{R}_j$. LCD includes two loop operations. The former is used to count the number of collisions between the prefix sub-chain η and all candidates of κ_j within threshold τ_{κ_j} (Steps 3 ~ 6). The latter is used to avoid the repetitive collision detection of chains with the same prefix.

Algorithm 1. Chain Construction in LCD Algorithm

Input: Chains $\mathcal{R}_{j-1}$, sub-chains flag $\mathcal{F}$, candidates ξ_j and thresholds τ_{κ_j} , number of collisions on chains “numC” and collision threshold τ_c .
Output: New chains $\mathcal{R}_j$, array $\mathcal{F}$ and numC.

```

1  $\beta \leftarrow 1, \gamma \leftarrow 1, \pi \leftarrow 0, \mathcal{F} \leftarrow 0;$ 
2 while  $\beta \leq |\mathcal{R}_{j-1}|$  and  $\gamma \leq |\mathcal{F}|$  do
3    $\eta \leftarrow \mathcal{R}_{j-1}^\beta;$ 
4   for  $\epsilon$  from 1 to  $\tau_{\kappa_j}$  do
5      $\mathcal{N}_\epsilon \leftarrow \sum_{i=1}^{j-2} \phi(\eta_i, \xi_j^\epsilon);$ 
6   end
7   for  $v$  from 0 to  $\mathcal{F}_\gamma - 1$  do
8      $\chi = 0;$ 
9     for  $\epsilon$  from 1 to  $\tau_{\kappa_j}$  do
10       $\pi ++; \chi ++;$ 
11       $\eta \leftarrow \mathcal{R}_{j-1}^{v+\beta};$ 
12       $\text{numC1}_\pi \leftarrow \text{numC}_{v+\beta} + \phi(\eta_{j-1}, \xi_j^\epsilon) + \mathcal{N}_\epsilon;$ 
13       $\mathcal{R}_j^\pi \leftarrow \mathcal{R}_{j-1}^{v+\beta} \bowtie \xi_j^\epsilon;$ 
14    end
15  end
16   $\beta \leftarrow \beta + \mathcal{F}_\gamma;$ 
17   $\gamma ++;$ 
18 end
19 remove chains dissatisfying given collision conditions;
```

For the current $\mathcal{F}_\gamma$ chains with the same prefix, we consider their collisions with the τ_{κ_j} candidates of κ_j respectively, and use χ to count the number of chains that meet the conditions (Step 8). We do not set collision conditions in this step for the convenience of further processing in Step 19 (also see Algorithm 2). The total number of chains π and χ increase by 1 when the ϵ -th candidate ξ_j^ϵ of κ_j is considered, and the current $(v + \beta)$ -th chain $\mathcal{R}_{j-1}^{v+\beta}$ is saved to η (Steps 10 ~ 11). We only need to detect the collision between its $(j-1)$ -th candidate and ξ_j^ϵ , and the total number of collisions of the new chain is $\text{numC}_{v+\beta} + \phi(\eta_{j-1}, \xi_j^\epsilon) + \mathcal{N}_\epsilon$ that finally saves to numC1_π (Step 12), thus achieving jump detection. The new chain $\mathcal{R}_{j-1}^{v+\beta} \bowtie \xi_j^\epsilon$ is saved as the π -th chain in $\mathcal{R}_j$ (Step 13). Here “ $\bowtie$ ” denotes the splicing that we add ξ_j^ϵ as the new last candidate of chain $\mathcal{R}_{j-1}^{v+\beta}$, and $\mathcal{N}_\epsilon$ denotes the number of collisions between the β -th chain in $\mathcal{R}_{j-1}$ and the ϵ -th candidate ξ_j^ϵ of κ_j (see Step 5). LCD then updates the indexes β and γ , and traverses the chains with the same next prefix sub-chain (Steps 16 ~ 17).

4.3 Chains Sieve

Finally, we sieve a part of chains satisfying the given collision conditions (as given in Algorithm 2), and finish the collision detection on sub-key κ_j . The corresponding parameters are updated accordingly to facilitate the collision detection of the next sub-key.

Algorithm 2. Chains Sieve in LCD Algorithm

```

1  $\gamma \leftarrow 0, \pi \leftarrow 0, \mathcal{F}' \leftarrow 0;$ 
2 for  $\beta$  from 1 to  $|\mathcal{R}_{j-1}|$  do
3    $\chi = 0;$ 
4   for  $\epsilon$  from 1 to  $\tau_{\kappa_j}$  do
5      $\pi = \tau_{\kappa_j} \cdot (\beta - 1) + \epsilon;$ 
6     if  $\text{numC1}_\pi \geq \tau_{c_j}$  then
7       copy  $\mathcal{R}_j^\pi, \text{numC1}_\pi$  to  $\mathcal{R}'_j$  and  $\text{numC1}'_\pi;$ 
8        $\chi ++;$ 
9     end
10  end
11  if  $\chi \geq 0$  then
12     $\gamma ++; \mathcal{F}'_\gamma = \chi;$ 
13  end
14 end
15 update numC,  $\mathcal{F}$  and  $\mathcal{R}$  using  $\text{numC1}'_\pi, \mathcal{F}'$  and  $\mathcal{R}';$ 
```

We store all combinations of the τ_{κ_j} best candidates of κ_j and $\mathcal{R}_{j-1}$ in Algorithm 1, which facilitates our deletion according to the collision conditions. We still use β and ϵ to denote the indexes of $\mathcal{R}_{j-1}$ and ξ_j , and compute the index π for each of their values in $\mathcal{R}_j$ and numC1 (Step 5 in Algorithm 2). If the corresponding number of collisions numC1_π is greater than the collision threshold τ_{c_j} , we copy the chain $\mathcal{R}_j^\pi$ and its corresponding number of collisions numC1_π to $\mathcal{R}'_j$ and $\text{numC1}'_\pi$ (Steps 6 ~ 9). We also exploit χ to record the number of chains with the same prefix sub-chain $\mathcal{R}_{j-1}^\beta$, and its value is accumulated accordingly (Step 8). It is detected to determine whether there is a chain prefixed with $\mathcal{R}_{j-1}^\beta$ after traversing the τ_{κ_j} best candidates of κ_j (Steps 11 ~ 13). Since the j th sub-key can collide with the previous $j-1$ sub-keys and has $j-1$ collisions at most, it may not be ideal if τ_c is set to a fixed value (e.g., $\tau_c = 4$). In other words, it

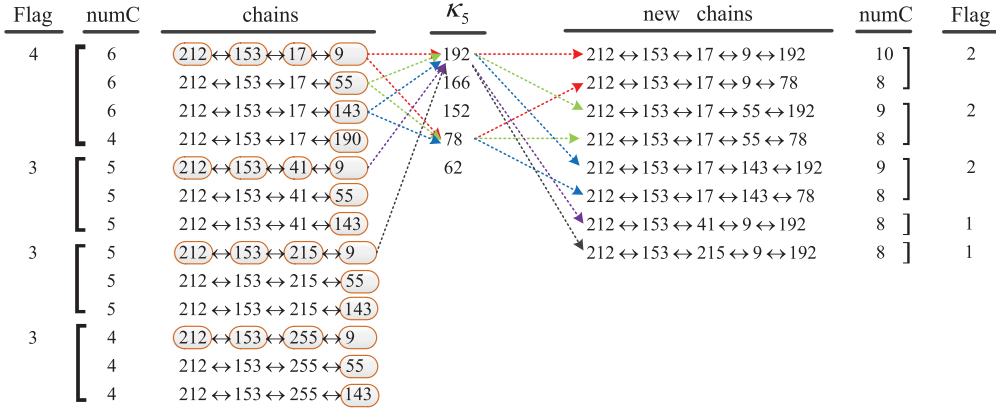


Fig. 2. Jump collision detection on the remaining chains of the first 4 sub-keys in LCD when considering the coming sub-key κ_5 .

may be too large for the first several sub-keys and too small for the last several sub-keys. We will further discuss its settings in Section 5.

4.4 An Example for Jump Collision Detection Mechanism

The partial results of one attack that LCD performs on 240 randomly selected traces are shown in Fig. 2. Here τ_d , τ_k and τ_c are set to 10, 5 and 3 respectively, and the array “Flag” denotes $\mathcal{F}$. The first 13 chains in $\mathcal{R}_4$ belong to 4 prefixes respectively. Our LCD first detects the collisions between the prefix chain $212 \leftrightarrow 153 \leftrightarrow 17$ and the 5 candidates of the sub-key κ_5 . Then, we just detect the collision between the 4 candidates of κ_4 on the first 4 chains and κ_5 , and directly skip 9 collision detections. The array $\mathcal{F}_1$ in Step 7 of Algorithm 1 has been satisfied and exits when traversing the 4th chain, so LCD continues to detect the collision between the 3 chains corresponding to the next prefix $212 \leftrightarrow 153 \leftrightarrow 41$ and candidates of κ_5 . This repetition finishes when the last chain in $\mathcal{R}_4$ in Fig. 2 has been detected, and a total of $13 \cdot 5$ chains are obtained (Algorithm 1 has reached its Step 19 here). Then, the array $\mathcal{F}$, $\mathcal{R}_5$ and “numC1” are updated using Algorithm 2 to obtain 8 chains that ultimately meet $\tau_c = 3$. Collision detection performed by our LCD is “sparse” as shown in Fig. 2. This sparsity is more obvious when LCD traverses to the last several sub-keys, which saves a lot of time for collision detection.

5 OPTIMIZATION

Our LCD algorithm exploits collisions between any two sub-keys, thus requiring fault tolerance on δ -s to guarantee its key recovery with a large probability under small threshold τ_d . Moreover, we also exploit guessing theory to optimize the collision detection order of sub-keys according to their number of candidates under a fixed probability, thus further reducing repetitive collision detections. Guessing theory also provides us a very good evaluation tool. Finally, it is difficult to set a reasonable τ_d , as too many chains remain if it is large. However, the key is usually one of the chains with almost the largest number of collisions. We further optimize the chain sieve by exploiting this fact. The above 3 optimizations will be introduced in the next subsections.

5.1 Fault Tolerance Strategies

Our LCD algorithm makes use of 120 pairs of collisions between any two sub-keys of AES-128, which are many more than 15 pairs exploited in TC and FTC, and 32 pairs exploited in GCA. Therefore, it can exploit a much larger candidate space. In other words, we can set a much larger threshold τ_d for LCD. For a sufficiently large threshold τ_d , all required δ -s are within it. However, it is difficult to achieve this, since some of them are poorly ranked. Therefore, we achieve the successful key recovery by tolerating some collision values δ -s outside the threshold rather than considering a larger τ_d .

We first consider the distribution of δ -s. We randomly extract 240 power traces to perform the MDCA given in Section 2.3, and repeat this operation 2,000 times. The number of δ -s corresponding to the second to the 16th sub-keys exceeding the threshold τ_d is shown in Fig. 3. Only a very small part of δ -s fall outside the small τ_d -s in most experiments. With the increase of τ_d , more δ -s fall within the threshold. To better illustrate this, we also randomly extract 240 power traces for MDCA and repeat this experiment 25 times. The results are shown in Fig. 4. They are a total of 18 repetitions with fewer than 20 δ -s beyond threshold τ_d . Most δ -s fall within this threshold under MDCA. In general, the difference in the number of δ -s between the correct chain and the chain with the largest number of collisions, is always kept in a small range.

A sub-key may have several corresponding δ -s beyond threshold, but the δ -s corresponding to the several neighboring sub-keys may fall within the threshold. In this way, τ_c of each sub-key can be appropriately large, but the total number of δ -s beyond threshold can be much smaller than the sum of them. Here we still use numC1_π to denote the number of collisions on the π -th chain. We can set a threshold:

$$\tau_{c_{k_j}} = \max(\text{numC1}) - \left\lceil \frac{j}{\varphi} \right\rceil, \quad (15)$$

flexibly for the j th sub-key in Step 6 of Algorithm 2. Here “ $\lceil \cdot \rceil$ ” denotes the round up operation, and the new symbol φ denotes threshold adjustment factor. In this case, each sub-key shares its τ_c with neighbors, and the total number of δ -s allowed to be beyond threshold between several adjacent sub-keys can be maintained in a small range. For TC and FTC, the long chains are constructed from the short ones having the same number of collisions, thus this

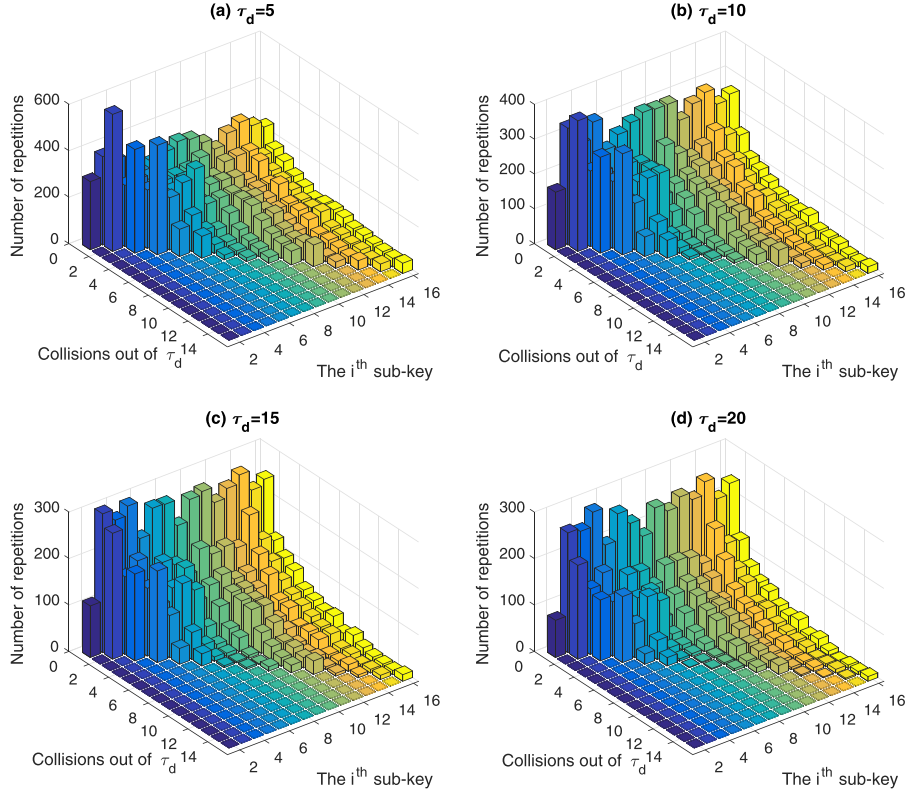


Fig. 3. Distribution of δ -s beyond different τ_d -s under average 240 traces and 2 000 repetitions.

adjustment does not exist. There is neither such a fault tolerance strategy, nor such an adjustment in GCA.

5.2 Guessing Theory Optimization

The performance of CODCAs such as TC, FTC, and GCA, can be compared through the success rate [22] under the same thresholds τ_κ and τ_d . However, currently there is no effective way of evaluating how good CODCAs are, i.e., how far is their performance from the ideal success rate. Here the guessing theory is introduced to provide a good reference as exploited in [16]. Specifically, a metric considers only the candidates that satisfy the given probability value $\alpha_j \in (0, 1)$:

$$\mu_{\alpha_j}(\mathbb{K}_j) = \min \left\{ \tau_{\kappa_j} \mid \sum_{i=1}^{\tau_{\kappa_j}} p_j^i \geq \alpha_j \right\}, \quad (16)$$

which was named as α -work-factor in [19].

In this case, the threshold τ_κ can be adjusted flexibly according to the probability distribution. The guessing space:

$$\Omega = \prod_{j=1}^{16} \mu_{\alpha_j}(\mathbb{K}_j), \quad (17)$$

and the theoretical success rate:

$$\vartheta = \prod_{j=1}^{16} \alpha_j, \quad (18)$$

can be easily obtained according to the guessing lengths $\mu_{\alpha_1}(\mathbb{K}_1), \dots, \mu_{\alpha_{16}}(\mathbb{K}_{16})$ and the corresponding success rates $\alpha_1, \dots, \alpha_{16}$ of 16 sub-keys in AES-128 algorithm. In this case, the thresholds τ_κ and τ_d can be adjusted reasonably

according to the computing power. Here we also set the threshold τ_c according to Eq. (15). To minimize the burden of collision detection, we can detect the sub-keys according to their number of candidates within τ_κ in ascending order as in [16]. More chains will be established if the sub-keys with more candidates are in the front. By simply adjusting the order of collision detection, the collision conditions of a certain sub-key become different. We set α and τ_d to 0.990 and 10 without considering τ_c , and randomly extract power traces 200 times. The average number of chains under different number of traces with or without key re-ranking is shown in Fig. 5, which indicates that our LCD achieves the smallest remaining chain space under the same α and τ_d after re-ranking.

5.3 Sieve Optimization

Cryptographic algorithms usually have a huge key candidate space to guarantee their security. The attacker needs to face the difficulty of key recovery in many scenarios includ-

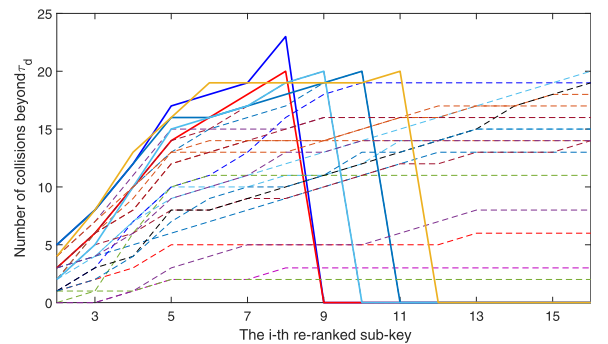


Fig. 4. Number of collisions beyond threshold $\tau_d=10$.

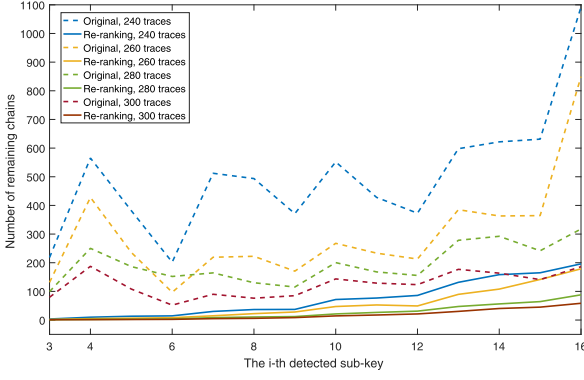


Fig. 5. Number of remaining chains under guessing entropy with or without re-ranking optimization.

ing side-channel attacks. Sieving enables the attacker to discard the remaining chains when the number of the guessed ones have exceeded a given threshold, which depends on his computing power. Actually, we set τ_k and τ_d to limit the guessing space of the improved TA and MDCA, and discard the chains with collisions fewer than τ_c , which are all sieves. However, collision detection may still face a large guessing space if these thresholds are unreasonably set. To address this issue, we can further construct a simple candidate sieve, which can be achieved by simply modifying Algorithm 1. Specifically, we execute Algorithm 3 before Algorithm 2.

Algorithm 3. Sieve Optimization in LCD Algorithm

```

1  $\Phi \leftarrow 0, \pi \leftarrow 0$ ;
2  $(\Psi, \Theta) \leftarrow \text{RankAscend}(\text{numC1})$ ;
3 for  $\beta$  from 1 to  $|\Psi|$  do
4   if  $\Psi_\beta \geq \tau_{c_j}$  then
5      $\pi \leftarrow \pi + 1$ ;
6   end
7   if  $\pi \geq \tau_n$  then
8     break;
9   end
10 end
11 transform collision locations into flags:  $\Phi_{\Theta_{1,\dots,\pi}} \leftarrow 1$ ;
```

First, we initialize a flag array Φ and exploit π to record the current number of chains (Step 1 in Algorithm 3). We rank chains according to their number of collisions in descending order, and store the numbers and indexes into Ψ and Θ , respectively (Step 2). Then, we detect the number of collisions in Ψ according to Θ in turn. If it is greater than the collision threshold τ_{c_j} (Condition 1), then the number of chains π satisfying the condition increases by 1. Here we also check whether π is greater than τ_n (Condition 2). In other words, we only consider the τ_n chains with the largest number of collisions. The loop operation stops if this condition is satisfied (Steps 3 ~ 10). Then, we transform the indexes of these chains that meet both conditions into flags and store them in the array Φ , namely " $\Phi_{\Theta_{1,\dots,\pi}} \leftarrow 1$ ". For example, if Θ_1 , the first element of Θ , satisfies the conditions, then we set the corresponding flag in Φ_{Θ_1} to 1. Finally, we only need to check whether the π -th chain provided by Step 5 in Algorithm 2 has a flag "1" in Φ , thus making the future collision detection convenient. We can

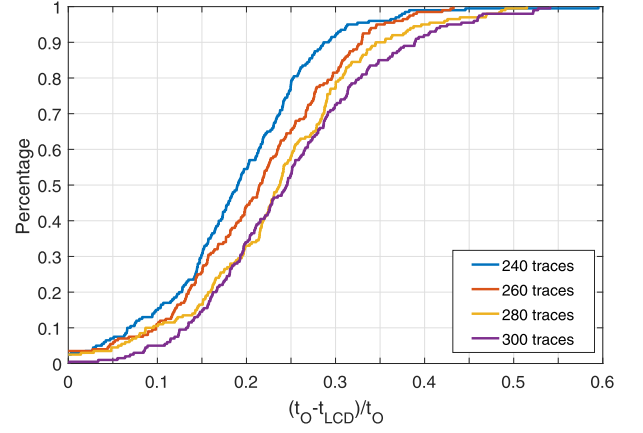


Fig. 6. Time consumption optimized by our LCD algorithm compared to detect the chains one by one without any repetitive detection avoidance.

see from Algorithms 1, 2 and 3 that our LCD is very simple. Its high efficiency will be validated by the experiments in Section 6.

6 EXPERIMENTAL RESULTS

6.1 Performance Improvement by Jump Collision Detection Mechanism

We first show the performance optimized by jump collision detection mechanism. We randomly extract different number of traces and repeat each experiment 300 times. Here τ_d and τ_c are set to 8 and 3, and α is set to 0.990 ($\tau_k = \mu_\alpha$). The percentage of time optimized is shown in Fig. 6. t_O and t_{LCD} on horizontal ordinate denote the time consumption of collision detection before and after LCD optimization. It is evident that our LCD saves about 15% ~ 40% of the time in most operations. Some of the experiments are particularly time-consuming and take several or even dozens of hours, wherein the performance of LCD is improved more significantly. We can also find that the gap between curves becomes larger when the number of samples used for attack in Fig. 6 becomes smaller. This fully illustrates that the larger the number of chains, the better the jump detection mechanism works.

TABLE 7
Success Rates Under Different Thresholds τ_d

τ_d	4	8	12
TC	0.025	0.090	0.165
FTC	0.005	0.050	0.130
GCA	0.005	0.015	0.045
LCD	0.515	0.600	0.640

TABLE 8
Time Consumption Under Different Thresholds τ_d (Seconds)

τ_d	4	8	12
TC	0.0110	0.0103	0.0107
FTC	0.0194	0.0306	0.1871
GCA	0.0217	0.0281	0.0435
LCD	0.8483	2.8876	5.4037

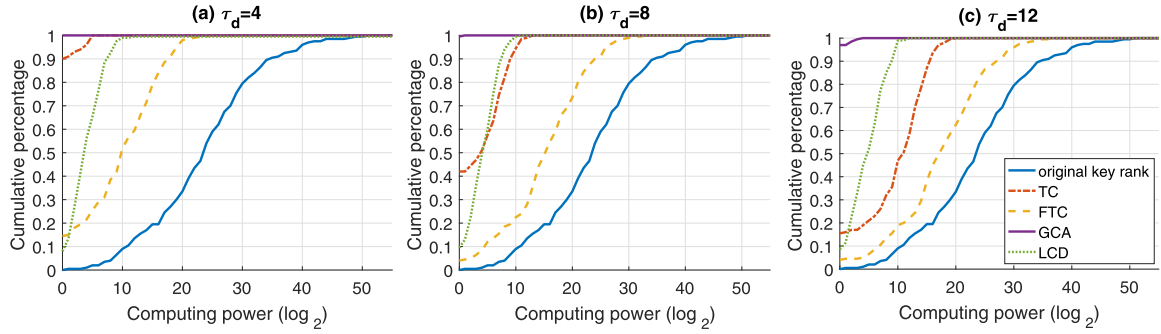


Fig. 7. Number of chains to be verified in key recovery under different thresholds τ_d from 4 to 12.

6.2 Experiments Under Different Thresholds τ_d

We then compare the performance of 4 CODCAs TC, FTC, GCA and our LCD under different thresholds τ_d -s. A total of 240 power traces are randomly extracted, and this operation repeats 200 times. Here the probability threshold α in guessing entropy introduced in Section 5.2 is set to 0.990, thus the overall probability of key recovery is $0.99^{16} \approx 0.8515$. LCD is a lightweight collision detection algorithm. Its threshold can be set very small, which cannot be achieved with existing schemes. Therefore, we set the threshold τ_d to 4, 8, 12 respectively, of which the latter two make their success rates easier to observe. Here we further set the threshold adjustment factor φ in Eq. (15) to 3, and τ_n in Algorithm 3 to 1,000, which means we only consider almost 1,000 chains having the largest number of collisions in our LCD, thus making the key recovery easy.

Success rates of the 4 CODCAs TC, FTC, GCA and LCD under different thresholds τ_d are shown in Table 7. Benefiting from the fault tolerance strategy, our LCD algorithm can achieve significantly higher success rate than other schemes even under a very small threshold. On one hand, although τ_d is very small, most of the collision values δ -s output by MDCA still fall within it, which enables the correct sub-keys to meet the collision conditions with a high probability and be preserved. On the other hand, due to the small thresholds, only a very small number of chains satisfy the collision conditions, thus the chain space is small, which is convenient for the later key recovery.

The CODCAs exploit the collision information to optimize the divide-and-conquer attacks, and transform a part of its best candidates into a much smaller chain space, from which the attacker can try to recover the key. We provide the

cumulative distribution of the full-key candidates to be verified (i.e., computing power) with regards to the 4 schemes TC, FTC, GCA and our LCD for collision detection under different thresholds (as shown in Fig. 7). The horizontal coordinate represents the number of chains that need to be verified. For example, this is $\log_2 1024 = 10$ when we have to verify 1,024 chains, i.e., the key is the 1024th best chain. The longitudinal coordinate represents the cumulative percentage.

It is worth noting that for CODCAs, whether the key is within the threshold is unknown, they need to verify the chains one by one. As such, the number of chains required for verification is the whole chain space if the key does not satisfy a chain. Its principle is similar to the classic key enumeration (e.g., [20]), in the sense that the enumeration of candidates is from the one with the largest probability to the one with the smallest probability. However, the difference is that the former enumerates the original space and tries to find the most probable combination from the remaining candidate space, while the latter enumerates the chains, which are already combinations of sub-keys candidates. As a result, the latter is usually much faster than the former. In addition, FTC and TC have no corresponding flexible threshold adjustment mechanism, which may leave us a very large chain space to enumerate. If the key is not within the space, they still need to enumerate the chains from the first to the last before stopping. In this way, the chain space expands rapidly with the increase of τ_d , which makes the key verification time-consuming.

Due to exploiting about 8 times as many collisions as TC and FTC, and about 4 times as many collisions as GCA, LCD consumes more time in collision detection (as shown in Table 8). It can also be seen from Fig. 7 that our LCD algorithm does not lead to large candidate space. Therefore, our

TABLE 9
Success Rates Under Different Number of Traces

Number of traces	180	200	220
TC	0.015	0.020	0.060
FTC	0.005	0.015	0.050
GCA	0.000	0.010	0.020
LCD	0.285	0.395	0.485
Number of traces	260	280	300
TC	0.145	0.150	0.245
FTC	0.105	0.150	0.160
GCA	0.025	0.035	0.075
LCD	0.660	0.720	0.760

TABLE 10
Time Consumption Under Different Number of Traces (Seconds)

Number of traces	180	200	220
TC	0.0669	0.0405	0.0296
FTC	0.1249	0.0783	0.0568
GCA	0.1511	0.0893	0.0664
LCD	21.012	12.250	4.4959
Number of traces	260	280	300
TC	0.0102	0.0084	0.0064
FTC	0.0172	0.0117	0.0079
GCA	0.0216	0.0155	0.0108
LCD	0.7816	0.6180	0.3179

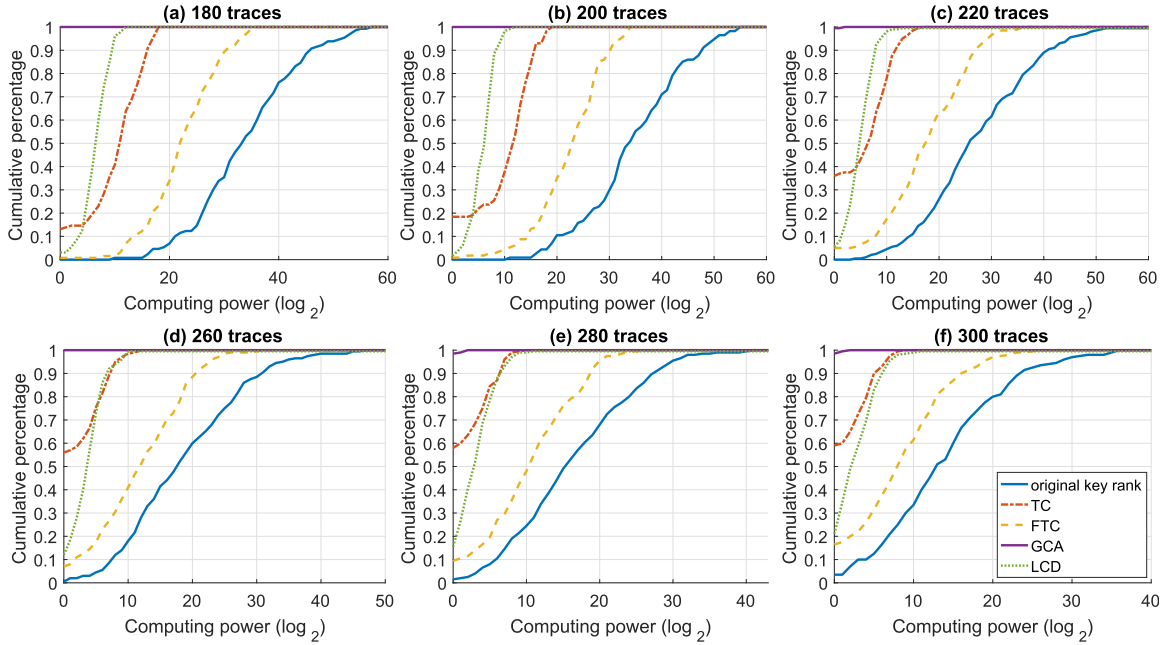


Fig. 8. Number of chains to be verified in key recovery under different number of traces.

scheme takes much less time than TC and FTC in the later key recovery. The number of chains to be verified in TC and FTC increases significantly compared to our LCD. FTC has a probability of 0.03 to enumerate more than 2^{20} chains when $\tau_d = 4$. This becomes 0.26 and 0.37 when τ_d reaches 8 and 12, respectively. The number of chains to be verified in FTC in some repetitions is even close to 2^{40} when $\tau_d = 12$. In fact, TC is also too time-consuming when constructing chains under $\tau_d = 16$. The number of chains under a certain sub-key is more than one million in many repetitions, and finally tens of thousands of them remain (this is also reflected in Fig. 7c). This clearly shows that TC and FTC cannot run against such a huge threshold. Therefore, we do not compare for much larger τ_d in this paper.

6.3 Experiments Under Different Number of Traces

We finally compare the performance of the 4 CODCAs (i.e., TC, FTC, GCA and LCD) under different number of traces. Here the threshold τ_d is set to 8 and the experimental results are shown in Table 8. With the increasing number of traces, more collision values δ -s fall within threshold τ_d and the success rates significantly improve. The success rate of our LCD shown in Table 9 is much higher than that of the existing schemes. It even achieves a significantly higher success rate under 160 traces than other methods under 300 traces. We also give the number of chains that need to be considered to recover the key under different number of traces in these 4 CODCAs (as shown in Fig. 8). These fully illustrate that our LCD not only achieves a higher success rate, but also significantly reduces the number of chains to be verified and alleviates the difficulty of key recovery.

We can compare the performance of the 4 CODCAs in pushing the rank of key by the cumulative percentage and success rate achieved under different number of chains to be verified by exploiting information from Tables 7 and 9, and Figs. 7 and 8. The key rank estimation on the original candidate space of the optimized TA is also given. Taking Fig. 8a and Table 9 as examples, the success rate of FTC

under 280 traces is about 0.150, and the number of chains to be verified is between 0 and 2^{24} . Similar with TC and GCA, FTC does not significantly optimize the rank of key and reduce the difficulty of key recovery. LCD achieves a success rate of 0.720 at 280 traces and 0.760 at 320 traces, which approaches the theoretical success rate $0.990^{16} \approx 0.8515$ as mentioned in Section 6.2.

It is worth noting that successful attacks do not mean more advanced key ranking. Actually, these CODCAs can detect very huge candidate spaces such as 20^{50} and reduce them to 2^{12} (as can be seen in Fig. 7c and Figs. 8a and 8b). The same conclusion on the reduction of candidate space has been discussed in detail in [17]. In other words, the “rank” of key can be significantly improved after transforming a part of the best candidates of divide-and-conquer attacks like the optimized TA exploited in this paper into chain space through CODCAs. Moreover, the “key enumeration” in chain space simply ranks the probabilities of chains from the largest to the smallest, and then verifies them in turn, which is simpler and more efficient than the traditional key enumeration. Although our LCD algorithm brings a few seconds of extra time overhead (see Table 10), it leaves a much smaller chain space compared to the existing schemes, saves significantly more time for future key recovery and achieves significantly higher success rate. The increasing number of power traces also reduces the number of candidates in the optimized TA, thus making the collision detection easier.

7 CONCLUSION

Recovering the key ranking in very deep space is still one of the major challenges faced by researchers in side-channel attacks. Limited to the enumeration power, the attacks are only practical for a space that is a fraction of the original one. By introducing collision information into divide-and-conquer attacks, the existing CODCAs transform a part of the best candidates, which may be too large to enumerate, into a significantly smaller collision chain space, so that key recovery is still

possible. However, they employ low efficiency or poor fault tolerance mechanism, which severely restricts the performance.

To solve the above problems, we proposed a novel lightweight collision detection algorithm named LCD in this paper. It exploited a jump detection mechanism to avoid the repetitive collision detection of chains with the same prefix sub-chains, which significantly improved the detection efficiency. We also introduced guessing entropy to optimize its search order of sub-keys and provide a good reference when evaluating the CODCAs. Finally, we designed a highly efficient fault tolerance mechanism for our LCD algorithm, thus enabling its flexible thresholds adjustment and quick sieving of the best chains. The experiments validated that our LCD not only achieved success rate that are significantly higher than the existing schemes, but it also results in the smallest chain space, which makes future key recovery easier.

ACKNOWLEDGMENTS

This work was supported in part by the National Research Foundation Singapore under its Campus for Research Excellence and Technological Enterprise (CREATE) Programme with the Technical University of Munich at TUMCREATE.

REFERENCES

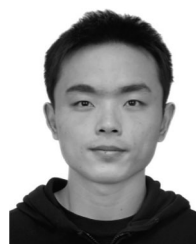
- [1] DPA contest, Accessed: Aug. 15, 2020. [Online]. Available: <http://www.dpacontest.org/home/>
- [2] A. Bogdanov, "Multiple-differential side-channel collision attacks on AES," in *Proc. 10th Int. Workshop Cryptogr. Hardware Embedded Syst.*, 2008, pp. 30–44.
- [3] A. Bogdanov and I. Kizhvatov, "Beyond the limits of DPA: Combined side-channel collision attacks," *IEEE Trans. Comput.*, vol. 61, no. 8, pp. 1153–1164, Aug. 2012.
- [4] E. Brier, C. Clavier, and F. Olivier, "Correlation power analysis with a leakage model," in *Proc. 6th Int. Workshop Cryptogr. Hardware Embedded Syst.*, 2004, pp. 16–29.
- [5] N. Bruneau, C. Carlet, S. Guilley, A. Heuser, E. Prouff, and O. Rioul, "Stochastic collision attack," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 9, pp. 2090–2104, Sep. 2017.
- [6] S. Chari, J. R. Rao, and P. Rohatgi, "Template attacks," in *Proc. 4th Int. Workshop Cryptogr. Hardware Embedded Syst.*, 2002, pp. 13–28.
- [7] M. O. Choudary and P. G. Popescu, "Back to massey: Impressively fast, scalable and tight security evaluation tools," in *Proc. 19th Int. Conf. Cryptogr. Hardware Embedded Syst.*, 2017, pp. 367–386.
- [8] T. Espitau, P. Fouque, B. Gérard, and M. Tibouchi, "Side-channel attacks on BLISS lattice-based signatures: Exploiting branch tracing against strongswan and electromagnetic emanations in micro-controllers," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1857–1874.
- [9] D. Genkin, A. Shamir, and E. Tromer, "RSA key extraction via low-bandwidth acoustic cryptanalysis," in *Proc. 34th Annu. Cryptology Conf. Advances Cryptol.*, 2014, pp. 444–461.
- [10] B. Gérard and F. Standaert, "Unified and optimized linear collision attacks and their application in a non-profiled setting," in *Proc. 14th Int. Workshop Cryptogr. Hardware Embedded Syst.*, 2012, pp. 175–192.
- [11] P. C. Kocher, "Timing attacks on implementations of diffie-hellman, RSA, DSS, and other systems," in *Proc. 16th Annu. Int. Cryptol. Conf. Advances Cryptology*, 1996, pp. 104–113.
- [12] Y. Li, S. Wang, Z. Wang, and J. Wang, "A strict key enumeration algorithm for dependent score lists of side-channel attacks," in *Proc. 16th Int. Conf. Smart Card Res. Advanced Appl.*, 2017, pp. 51–69.
- [13] A. Moghimi, G. Irazoqui, and T. Eisenbarth, "CacheZoom: How SGX amplifies the power of cache attacks," in *Proc. 19th Int. Conf. Cryptogr. Hardware Embedded Syst.*, 2017, pp. 69–90.
- [14] A. Moradi, O. Mischke, and T. Eisenbarth, "Correlation-enhanced power analysis collision attack," in *Proc. 12th Int. Workshop Cryptogr. Hardware Embedded Syst.*, 2010, pp. 125–139.
- [15] C. Ou and S. Lam, "The science of guessing in collision-optimized divide-and-conquer attacks," *IACR Cryptol. ePrint Arch.*, vol. 2019, 2019, Art. no. 13.
- [16] C. Ou, S.-K. Lam, and G. Jiang, "Multiple-differential mechanism for collision-optimized divide-and-conquer attacks," *IACR Cryptol. ePrint Arch.*, vol. 2019, 2019, Art. no. 690.
- [17] C. Ou, Z. Wang, D. Sun, and X. Zhou, "Group collision attack," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 4, pp. 939–953, Apr. 2019.
- [18] C. Ou, Z. Wang, D. Sun, X. Zhou, and J. Ai, "Group verification based multiple-differential collision attack," in *Proc. 18th Int. Conf. Inf. Commun. Secur.*, 2016, pp. 145–156.
- [19] J. O. Pliam, "On the incomparability of entropy and marginal guesswork in brute-force attacks," in *Proc. 1st Int. Conf. Cryptol. India*, 2000, pp. 67–79.
- [20] R. Poussier, F. Standaert, and V. Grosso, "Simple key enumeration (and rank estimation) using histograms: An integrated approach," in *Proc. 18th Int. Conf. Cryptogr. Hardware Embedded Syst.*, 2016, pp. 61–81.
- [21] K. Schramm, G. Leander, P. Felke, and C. Paar, "A collision-attack on AES: Combining side channel- and differential-attack," in *Proc. 6th Int. Workshop Cryptogr. Hardware Embedded Syst.*, 2004, pp. 163–175.
- [22] F. Standaert, T. Malkin, and M. Yung, "A unified framework for the analysis of side-channel key recovery attacks," in *Proc. 28th Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2009, pp. 443–461.
- [23] N. Veyrat-Charvillon, B. Gérard, and F. Standaert, "Security evaluations beyond computing power," in *Proc. 32nd Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2013, pp. 126–141.
- [24] D. Wang, A. Wang, and X. Zheng, "Fault-tolerant linear collision attack: A combination with correlation power analysis," in *Proc. 10th Int. Conf. Inf. Secur. Pract. Experience*, 2014, pp. 232–246.
- [25] W. Wang, Y. Yu, F. Standaert, J. Liu, Z. Guo, and D. Gu, "Ridge-based DPA: Improvement of differential power analysis for nano-scale chips," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 5, pp. 1301–1316, May 2018.
- [26] A. Wiemers and D. Klein, "Entropy reduction for the correlation-enhanced power analysis collision attack," in *Proc. 13th Int. Workshop Secur.*, 2018, pp. 51–67.
- [27] H. Zhang and Y. Zhou, "How many interesting points should be used in a template attack?" *J. Syst. Softw.*, vol. 120, pp. 105–113, 2016.



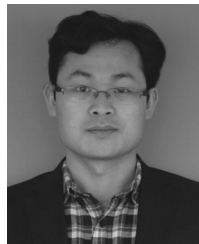
His current research interests include hardware and embedded system security, side-channel attacks, machine learning, and signal processing.



Siew-Kei Lam received the BSc, MEng, and PhD degrees from the School of Computer Engineering (SCE), NTU. His PhD dissertation investigated techniques to automatically generate custom instructions for reconfigurable processors. He was a visiting research fellow with the Imperial College of London, University of Warwick, and RWTH Aachen, Germany. He is currently an assistant professor with the School of Computer Engineering and his research interest includes investigates methods for realizing custom computing solutions in embedded systems.

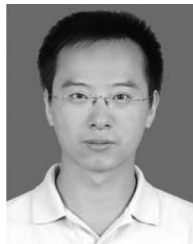


Chengju Zhou received MS degree from the School of Computer Science and Technology, Tianjin University, China, in 2015. He is currently working toward the PhD degree with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. His current research interests include focuses on AI security, signal processing, computer architecture, and object detection for urban traffic scene understanding.



Guiyuan Jiang received the BS degree in computer science and technology from Northwest University for Nationalities, China, the MEng degree in computer science and technology from Tianjin Polytechnic University, China, and the doctoral degree in computer science and technology from Tianjin University (TJU). Currently, he works as a research fellow with the Hardware & Embedded Systems Lab (HESL), School of Computer Science and Engineering, Nanyang Technological University (NTU), Singapore. His research interests

include AI security, data analysis, data driven optimization in intelligent transportation system, methodologies for designing secure and energy-efficient MPSoCs.



Fan Zhang received the PhD degree from the Department of Computer Science and Engineering, University of Connecticut, Storrs, CT, in 2012. He is an associate professor with the School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University, and affiliated with College of Information Science and Electronic Engineering, Zhejiang University. His research interests include hardware security, system security, cryptography, and computer architecture.

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**